MealMate.

End-term defense — practical results from Weeks 6–9:

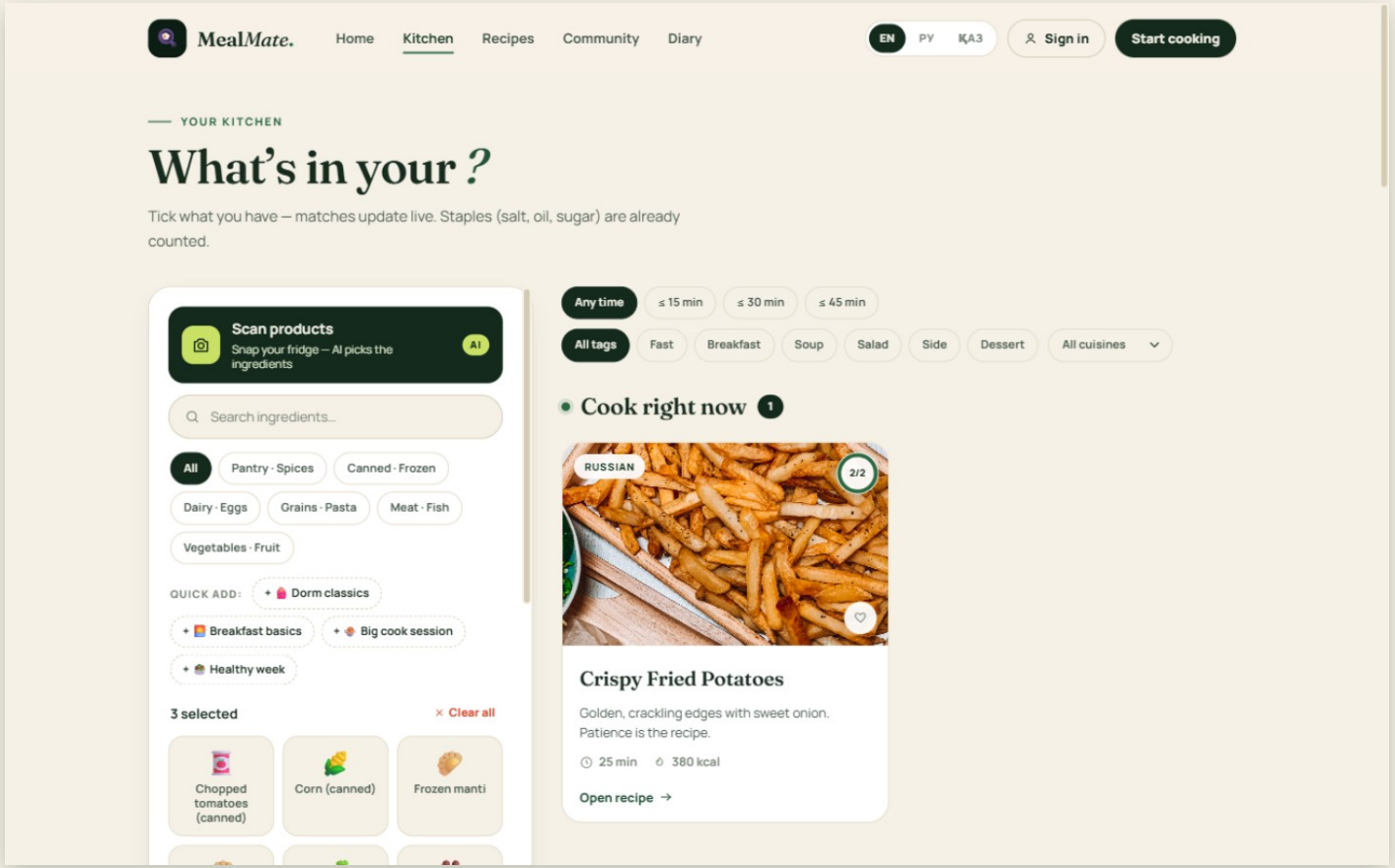
Database Systems · Cybersecurity · Multimedia Technologies

https://mealmate-29tt.onrender.com

CS-2618 · Samarin · Kaniyazov · Sadybek · Temirtas



A web app that matches recipes to the food you already own.



Live: the ingredient picker with an instant match list

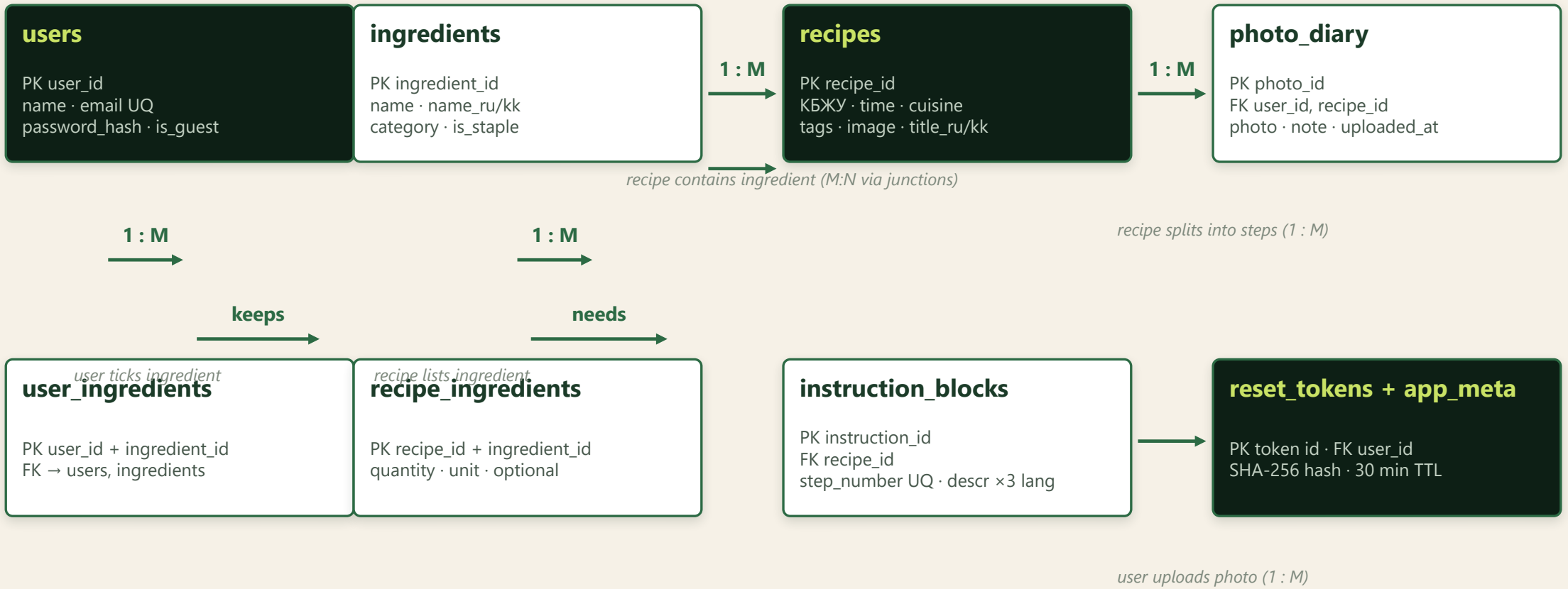
68 recipes across 5 cuisines — 14 Kazakh

51 picker ingredients, every one covered by a recipe

3 languages: EN / RU / KK — UI, content and cooking steps

Stack: React 18 → Express (Node 24) → SQLite. Deployed on Render, source on GitHub.

7 core entities, PK/FK on every table, verbs on relationships.



M:N resolved with junction tables; **editable source: docs/er-diagram.dbml → paste at dbdiagram.io during the defense.**

Business rules: unique e-mail · step numbers unique within a recipe · FK ON DELETE CASCADE.

Ready for the live challenge: change one thing, explain everything.

is_staple flag	salt, oil, sugar are assumed available — the matcher never counts them as missing. Change during defense: flip it and «Almost there» grows by hundreds of rows.
i18n columns	title_ru/kk, description_ru/kk on recipes; name_ru/kk on ingredients; localized steps. The API picks the column with ?lang= and falls back to English.
Junction extras	recipe_ingredients carries quantity, unit, optional — so the UI shows «600 g» and skips optional items when matching.
Indexes & integrity	FK ON DELETE CASCADE everywhere; indexes on junctions and diary(user_id); UNIQUE email and (recipe_id, step_number).

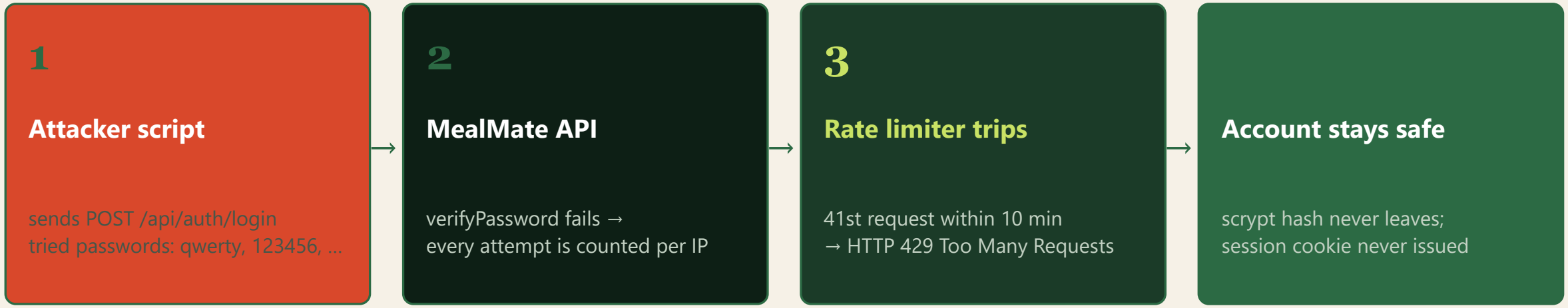
Matching query runs in SQL (two CTEs): missing = total − have per recipe, have ≥ 1 guarantees relevance — verified against the live deployment.

Five realistic risks — each closed by a measure that is actually in the code.

Risk	Consequence for users / data	Protection (implemented)
Brute-force password attacks on /login	Account takeover; personal diary exposed	Per-IP rate limit 40 req/10 min on auth; scrypt hashing makes offline cracking slow
SQL injection via search and filters	Full database dump or destruction	Parameterized queries only (node:sqlite) — zero string concatenation in SQL
XSS through diary notes and names	Script runs in other users' browsers, sessions stolen	React auto-escaping + strict server-side validation (type, length, MIME whitelist)
Session hijacking / cookie theft	Attacker acts as the victim	httpOnly + SameSite cookie, HMAC-SHA256 signature — unreadable and unforgable in JS
Oversized image uploads (DoS)	Disk and memory exhausted, service down	6 MB body limit, browser-side canvas compression to JPEG ≤ ~3 MB, MIME check

Also: security headers (helmet), one-time hashed password-reset tokens (30 min TTL), guest diary writes rejected with 401 at API level.

Threat: an attacker brute-forces the login. Watch the protection respond.

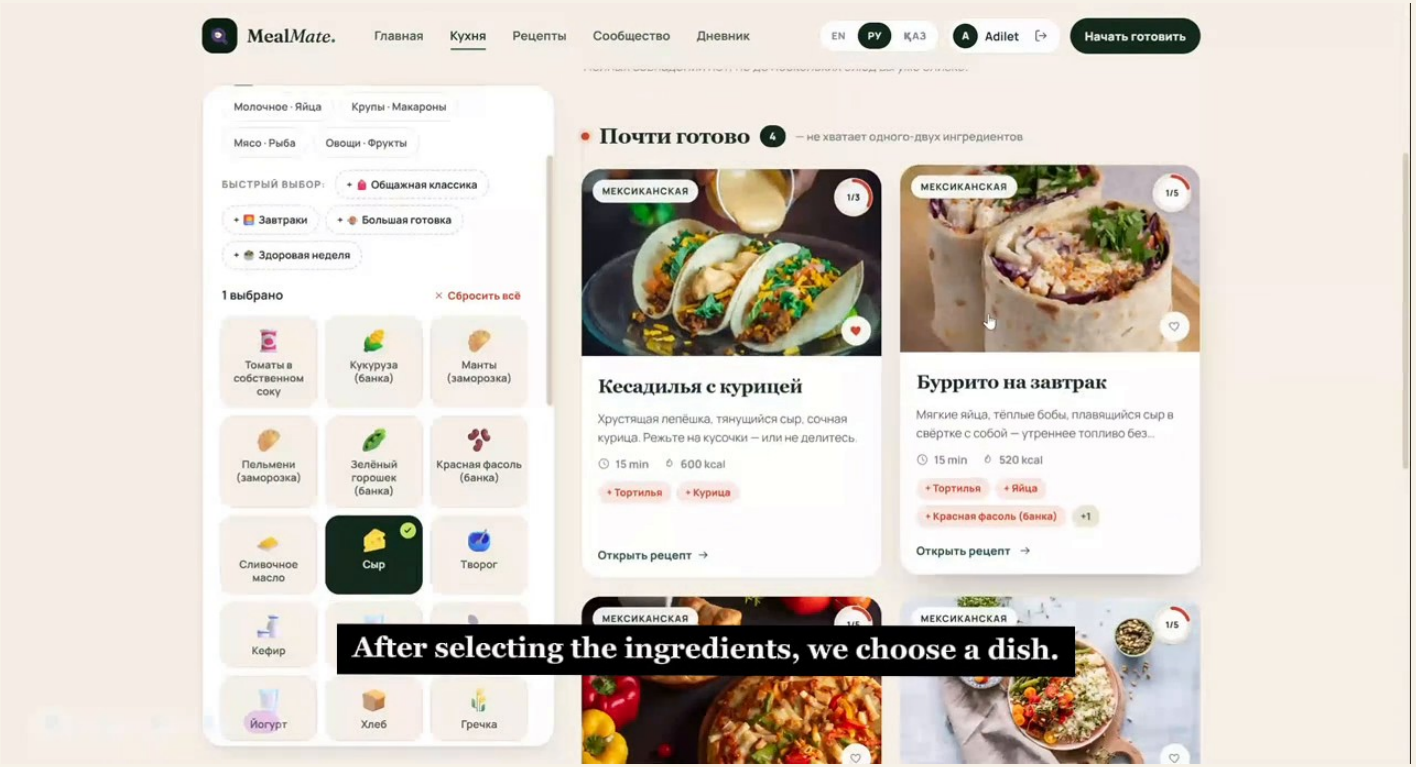


Live demo path: send 45 rapid wrong logins from the demo script → the 41st receives 429 and the rate bucket resets after 10 minutes. The same limiter protects diary uploads (20 / 10 min).

Why this scenario is safe: it runs against our own local deployment with a test account — no real third-party systems are touched.

Bonus hardening: scrypt makes each guess cost ~100 ms of CPU — brute-forcing a 8-char password becomes computationally absurd even offline.

A 1:37 promo, recorded by the team on the live deployed app.



▶ *Embedded video — plays directly from this slide during the defense*

3+ MULTIMEDIA ELEMENTS

Screen recording

Original team-made capture of the live deployed app — the required original element

Kinetic text captions

English subtitles burned into the footage, walking through each scene

Product footage

real flows on the deployment: catalog, matching, recipe pages in RU and EN

Audio

original synthesized soundtrack, added in post-production (source capture was silent)

INDIVIDUAL DEFENSE MAP

Every member owns a layer — ask us anything.

Artur — Lead	problem → solution reasoning, user research, feature priorities
Arsen — Frontend	design system, picker & matching UI, i18n, rendering pipeline, this deck
Adilet — Database	ER model, SQL matching CTEs, integrity, coverage audit
Sanzhar — Backend	Express API, auth & security, seed content, deployment

Live app https://mealmate-29tt.onrender.com **Source** github.com/nafiks/mealmate

AI disclosure: AI assisted code generation and deck design; every mechanism shown is deployed, verified and explained by the team.